

Cluster:

✓ Question 19 Correct

You are currently working with the application team to set up a SQL Endpoint. Once the team started consuming the SQL Endpoint, you noticed that during peak hours, as the number of concurrent users increases, query performance degrades and the same queries take longer to run. Which of the following steps can be taken to resolve the issue?

- ☐ They can turn on the Auto Stop feature for the SQL endpoint.

Explanation

Incorrect. Auto Stop controls idle resource termination, not performance under load.

- ☐ They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy from "Cost optimized" to "Reliability Optimized."

Explanation

Incorrect. Changing spot instance policy impacts reliability, not the number of concurrent queries handled.

Your answer is correct

- ☒ They can increase the maximum bound of the SQL endpoint's scaling range.

Explanation

Correct. Increasing the max scaling range adds more clusters, helping with query concurrency during peak usage.

- ☐ They can turn on the Serverless feature for the SQL endpoint.

Explanation

Incorrect. Serverless can reduce startup latency but is not the core solution to improve throughput for concurrent users.

- ☐ They can increase the cluster size(2X-Small to 4X-Large) of the SQL endpoint.

Explanation

Incorrect. Increasing cluster size (scale-up) improves individual query performance, not concurrency handling.

Overall explanation

To improve performance during high concurrency, increase the maximum bound of the scaling range on the SQL endpoint. This enables Databricks to spin up additional clusters to handle queued queries, improving throughput.

Domain

Databricks SQL

Overall explanation

A job cluster is the best choice for scheduled jobs because it terminates after execution, reducing costs compared to an all-purpose cluster.

You worked with the Data Analysts team to set up a SQL Warehouse so they can easily query and analyze data in the Gold layer. However, during peak hours, as the number of users increases, queries are taking longer to finish. Which of the following steps can be taken to resolve the issue?

Note: Databricks recently renamed SQL Endpoint to SQL Warehouse.

- ☐ They can turn on the Serverless feature for the SQL Warehouse and change the Spot Instance Policy from 'Cost Optimized' to 'Reliability Optimized.'

Explanation

Incorrect. Serverless and reliability-optimized policies can improve cost and stability, but they do not directly solve high query concurrency issues.

- ☐ They can turn on the Serverless feature for the SQL Warehouse.

Explanation

Incorrect. While serverless can help in some cases, increasing the scaling range is a more effective solution for handling high concurrency.

- ☐ They can increase the cluster size from 2X-Small to 4X-Large for the SQL Warehouse.

Explanation

Incorrect. Increasing cluster size (scaling up) improves single-query performance but does not resolve query queuing issues caused by high concurrency.

Your answer is correct

- ☒ They can increase the maximum bound of the SQL Warehouse's scaling range.

Explanation

Correct. Increasing the scaling range allows additional clusters to be added, reducing query queuing and improving response times

✓ Question 2 Correct

Which of the following describes a scenario in which a data team will want to utilize cluster pools?

Your answer is correct

- ☒ An automated report needs to be refreshed as quickly as possible.
- ☐ An automated report needs to be tested to identify errors.
- ☐ An automated report needs to be version-controlled across multiple collaborators.
- ☐ An automated report needs to be runnable by all stakeholders.

Overall explanation

Cluster pools in Databricks are used to ensure that a set of pre-warmed clusters is readily available to run workloads. This means that when a job is submitted, it can be executed more quickly because there is no need to wait for a cluster to spin up. Therefore, if a data team needs to refresh an automated report as quickly as possible, they will want to utilize cluster pools to ensure that the job can be executed as quickly as possible.

[For more information](#)

Question 30 **Incorrect**

A data engineering team has noticed that their Databricks SQL queries are running too slowly when they are submitted to a non-running SQL endpoint. The data engineering team wants this issue to be resolved.

Which of the following approaches can the team use to reduce the time it takes to return results in this scenario?

☐ They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized."

☐ They can turn on the Auto Stop feature for the SQL endpoint.

Your answer is incorrect

☒ They can increase the cluster size of the SQL endpoint.

Correct answer

☐ They can turn on the Serverless feature for the SQL endpoint.

Overall explanation

By enabling the **Serverless** feature for the SQL endpoint in Databricks, the team can reduce the startup time because Serverless endpoints are optimized for quick query execution without the need to manage clusters. Serverless architecture automatically manages infrastructure and resources, allowing queries to run almost immediately, even if the endpoint wasn't running when the query was submitted.

[For more information](#)

❗ **Question 21 Incorrect**

You noticed that a team member started using an all-purpose cluster to develop a notebook and used the same all-purpose cluster to set up a job that runs every 30 minutes to update underlying tables used in a dashboard. What would you recommend for reducing the overall cost of this approach?

Correct answer

- ☐ **Change the cluster all-purpose to job cluster when scheduling the job**

Explanation

Correct. Job clusters have significantly lower DBU costs than all-purpose clusters and are best for scheduled, non-interactive workloads.

- ☐ **Enable auto termination after 30 mins**

Explanation

Incorrect. Auto termination helps avoid idle costs but doesn't reduce DBU rate of an all-purpose cluster.

Your answer is incorrect

- ☒ **Change the cluster mode from all-purpose to single-mode**

Explanation

Incorrect. There is no 'single-mode' cluster option in Databricks; switching to a job cluster is the right approach.

- ☐ **Reduce the number of nodes and enable auto scale**

Explanation

Incorrect. Auto scaling helps with compute efficiency but doesn't change DBU rates.

- ☐ **Reduce the size of the cluster**

Explanation

Incorrect. Reducing cluster size may save some cost, but does not address the higher DBU cost of all-purpose clusters.

Overall explanation

Job clusters are optimized for scheduled jobs and incur lower DBU charges compared to all-purpose clusters. To reduce costs, schedule jobs using job clusters instead of long-running interactive clusters.

❗ **Question 31 Incorrect**

A data engineer has a Job with multiple tasks that runs nightly. Each of the tasks runs slowly because the clusters take a long time to start.

Which of the following actions can the data engineer perform to improve the start up time for the clusters used for the Job?

Your answer is incorrect

- ☒ They can use endpoints available in Databricks **SQL**

Explanation

Endpoints in Databricks SQL are used for connecting to external data sources and do not directly impact the start-up time of clusters. They are not relevant to improving the performance of cluster start-up time for the Job.

Correct answer

- ☐ They can use clusters that are from a cluster pool

Explanation

Using clusters from a cluster pool can significantly improve the start-up time for the Job tasks. Cluster pools allow for pre-created and pre-configured clusters to be used, reducing the time needed to start the clusters before running the tasks. This option directly addresses the slow start-up time issue for the Job.

- ☐ They can configure the clusters to autoscale for larger data sizes

Explanation

Configuring the clusters to autoscale for larger data sizes may help in handling varying workloads and data sizes efficiently, but it may not directly impact the start-up time of the clusters. It is more focused on resource allocation based on workload demands rather than improving the initial start-up time of the clusters.

- ☐ They can use jobs clusters instead of all-purpose clusters

Explanation

Using jobs clusters instead of all-purpose clusters may help in managing and organizing the tasks within the Job, but it does not directly address the issue of slow cluster start-up time. It is not a solution to improve the performance of cluster start-up time for the Job.

- ☐ They can configure the clusters to be single-node

Explanation

Configuring the clusters to be single-node may reduce the start-up time as fewer resources need to be initialized, but it may not be the most efficient solution for improving the overall performance of the Job. It may limit the scalability and processing power of the clusters.

Resources

Question 2 Incorrect

An engineering manager wants to monitor the performance of a recent project using a Databricks **SQL** query. For the first week following the project's release, the manager wants the query results to be updated every minute. However, the manager is concerned that the compute resources used for the query will be left running and cost the organization a lot of money beyond the first week of the project's release. Which of the following approaches can the engineering team use to ensure the query does not cost the organization any money beyond the first week of the project's release?

- ☐ They cannot ensure the query does not cost the organization money beyond the first week of the project's release.

Explanation

This is a false statement. The query can be managed effectively with the right scheduling options, such as setting an end date for the refresh schedule, which can help ensure that the query stops after the desired period, controlling costs.

- ☐ They can set a limit to the number of individuals that are able to manage the query's refresh schedule.

Explanation

Setting a limit to the number of individuals who can manage the query's refresh schedule does not directly address the issue of cost beyond the first week of the project's release. It may help control access to the query's configuration but does not stop the query from running and incurring costs.

Correct answer

- ☒ They can set the query's refresh schedule to end on a certain date in the query scheduler.

Explanation

Setting the query's refresh schedule to end on a certain date in the query scheduler is the correct approach to ensure that the query does not cost the organization any money beyond the first week of the project's release. By specifying an end date for the refresh schedule, the query will automatically stop running after the designated time, preventing any additional costs.

- ☐ They can set the query's refresh schedule to end after a certain number of refreshes.

Explanation

Setting the query's refresh schedule to end after a certain number of refreshes may help control the frequency of updates, but it does not guarantee that the query will stop running and consuming resources after the first week of the project's release. This approach may not effectively address the cost concerns beyond the initial period.

Your answer is incorrect

- ☒ They can set a limit to the number of DBUs that are consumed by the SQL Endpoint.

Explanation

Setting a limit to the number of DBUs consumed by the SQL Endpoint does not directly address the concern of cost beyond the first week of the project's release. It may help control costs related to the query's execution, but it does not ensure that the query does not continue

The data analysis team is experiencing slow performance with their Databricks **SQL** queries when using an always-on SQL endpoint. This issue arises when multiple team members run small queries simultaneously. After reviewing the system, the data engineering team identifies that all queries are being processed through the same SQL endpoint.

Which of the following actions would most effectively improve query performance under these conditions?

- ☐ **Switch the SQL endpoint to Serverless mode for automatic resource scaling.**

Explanation

1. **Explanation:** Serverless SQL endpoints in Databricks automatically scale based on the workload. They allow for dynamic allocation of resources as needed and are highly efficient in managing multiple simultaneous queries without requiring manual intervention. Serverless endpoints are particularly beneficial in scenarios with variable loads or when there is a need for quick response times for small queries from multiple users.
2. **Why it's not the best option:** Although this is a great solution for automatic scaling, **increasing the maximum bound of the scaling range** is more specific to the problem described in your case, where many users are running queries simultaneously.

Correct answer

- ☐ **Increase the maximum scaling capacity of the SQL endpoint to handle more concurrent queries.**

Explanation

This option is correct because increasing the maximum scaling capacity of the SQL endpoint allows Databricks to automatically allocate more resources to handle a higher number of concurrent queries, directly addressing the performance issue caused by simultaneous small queries. This improves query latency by ensuring sufficient compute resources are available.

- ☐ **Enable Serverless mode and configure the Spot Instance Policy to "Reliability"**

Somepoints:

Explanation

This statement is correct. When a file is created and stored in Databricks File System (DBFS), the file is actually persisted in the underlying cloud storage, such as Google Cloud Storage, AWS S3 bucket, or Azure Blob Storage. This allows for data durability and accessibility across different compute instances.



When we deploy a databricks workspace it is deployed in the control plane of the lakehouse platform.

Explanation

This statement is correct. When a Databricks workspace is deployed, it is provisioned in the control plane of the lakehouse platform. The workspace provides a collaborative environment for data engineering and data science tasks.

Correct answer



When we deploy a Spark cluster, the compute will be in the control plane and the storage will be in the data plane of the lakehouse platform.

Explanation

This statement is incorrect. In a Spark cluster deployment, both compute and storage resources are managed in the data plane of the lakehouse platform.



When the cluster is terminated all the data is lost from the cloud storage.

❗ Question 39 **Incorrect**

Which two methods are used by Auto Loader to identify new files and incremental data in Cloud object storage?

☐ Write-ahead logging, Read-ahead logging

☐ File listing, Dynamic file lookup

Your answer is incorrect

☒ Checkpointing, Write-ahead logging

☐ Checkpointing, Idempotent sinks

Correct answer

☐ Directory listing, File notification

Overall explanation

Auto Loader can detect new files in two ways: **directory listing** and **file notification**.

1. Directory Listing Mode

- Auto Loader looks at the files in the directory to find new ones.
- It's easy to set up because you only need access to the data in cloud storage, with no extra configuration.
- In newer versions of Databricks (9.1+), Auto Loader can detect file arrival order, reducing the number of API calls needed.

2. File Notification Mode

- Auto Loader uses cloud services to get notifications when new files are added.
- This mode is faster and better for large directories with many files.
- However, it needs extra cloud permissions to set up.

Resources

➡ [Auto Loader file detection modes](#)

Overall explanation

- **Delta Live Tables (DLT)** can run in **two modes: Continuous Mode** and **Triggered Mode**.
 - **Continuous Mode** means that the pipeline continuously processes new data as it arrives, which aligns with the behavior of keeping the compute resources active for rapid testing.
 - **Triggered Mode** involves batch processing at specified intervals rather than continuously running.
- **Development settings** typically involve running pipelines in continuous mode for rapid testing and debugging, allowing for the constant ingestion and processing of data. These settings do not require long-running production-like stability, which fits the description provided.
- **Production settings** generally focus on stability, so they would more likely use **Triggered Mode**, where the pipeline is triggered at fixed intervals, allowing for more controlled operations.

Given that the pipeline is designed for rapid testing and remains active (which suggests continuous operation), and considering that it hasn't been explicitly stated as a production environment, **Development settings** are the most appropriate.

Overall explanation

Change Data Capture (CDC) in the context of Delta Lake refers to the process of capturing changes made to data over time, such as inserts, updates, and deletes. The key idea behind CDC is that it enables you to track and capture only the changes that occur, rather than reprocessing the entire dataset every time a new batch of data is ingested.

Here's how CDC works in Delta Lake:

1. **Tracking Changes:** CDC identifies the changes that happen to the data over time (new records, updated records, or deleted records). This is typically done using some form of timestamp or versioning mechanism.
2. **Incremental Processing:** Once changes are identified, only the new or modified data is processed. This is efficient because it avoids the need to reprocess the entire dataset, saving time and resources. This method is essential in real-time data processing or when dealing with large datasets where reprocessing everything would be expensive.
3. **Using Delta Lake Features:** Delta Lake's ACID transactions, schema evolution, and time travel features make it a good fit for implementing CDC. For instance, Delta Lake's **merge** operation can be used to efficiently perform upserts (insertions and updates) in the data, ensuring the latest data is stored without duplication.

Resources

DLT:

❗ Question 12 **Incorrect**

Which of the following is a key benefit of using Delta Live Tables (DLT) for incremental data processing in Databricks?

- ☐ It performs full data reloads at regular intervals to maintain up-to-date datasets.

Explanation

One of the key benefits of Delta Live Tables (DLT) is that it performs **incremental** data processing, not full data reloads. DLT is designed to process only the new or changed data (using features like Change Data Capture or CDC), which is more efficient than reloading the entire dataset. Full reloads would negate the advantages of incremental processing.

Correct answer

- ☐ **It allows for automatic handling of schema evolution and data quality issues during incremental processing.**

Explanation

This is a key benefit of using Delta Live Tables (DLT) for incremental data processing in Databricks. DLT allows for automatic handling of schema evolution and data quality issues during incremental processing, making it easier to manage changes in data structures and ensure data quality.

- ☐ It automatically partitions data based on the timestamp column for faster querying.

Explanation

Delta Live Tables (DLT) does not automatically partition data based on the timestamp column for faster querying. While Delta tables do support partitioning, this specific benefit is not associated with using DLT for incremental data processing.

Your answer is incorrect

- ☒ **It eliminates the need for manual data validation steps during pipeline execution.**

Explanation

Delta Live Tables (DLT) does not eliminate the need for manual data validation steps during pipeline execution. Data validation is still an essential part of the data engineering process, and DLT does not replace this step.

- ☐ It enables the use of external tables for incremental processing, bypassing Delta tables entirely.

Explanation

Delta Live Tables (DLT) works specifically with Delta tables and doesn't bypass them entirely in favor of external tables for incremental processing. While DLT can process data from external sources, the main focus is on working with Delta Lake's optimized tables. External tables like Parquet or CSV are generally not the focus of DLT's incremental processing features.

- **Auto Loaders can split the processing into multiple batches so it is more efficient at scale.**

Explanation

Auto Loaders are capable of splitting the processing of data into multiple batches, which improves efficiency when dealing with a large volume of files. This capability allows for parallel processing and faster ingestion at scale.

Overall explanation

While the COPY INTO method can be used for loading data, it is typically more suited for smaller batch loads rather than handling millions of files efficiently compared to Auto Loader, which is designed specifically for large-scale ingestion and can handle incremental loading more effectively.

~

Overall explanation

In the Databricks architecture:

- **Control Plane:** This layer manages the Databricks workspace, including job scheduling and cluster management.
- **Compute/Data Plane:** This is where the actual data processing occurs, and it typically includes the compute resources (like Spark clusters) and the data storage (such as data stored in cloud storage).

When a Spark cluster is deployed, the compute resources (the cluster) operate within the **data plane**, not the control plane, while the control plane manages the orchestration and scheduling.

~

Overall explanation

While Delta Lake is indeed an open-source technology and a storage layer on top of a data lake that allows for ACID transactions, it is more accurately described as a storage layer or an optimization layer that adds functionality to existing storage formats (like Parquet) rather than being a standalone storage format itself.

To clarify the other points:

- **Open Source Technology:** Delta Lake is open source and available for use in various environments.
- **Storage Layer:** It acts as a storage layer that enhances data lakes by providing capabilities like ACID transactions, scalable metadata handling, and unifying batch and streaming data processing.
- **Runs on Apache Spark:** Delta Lake runs on top of Apache Spark, leveraging its capabilities for big data processing.

Overall explanation

By default, Delta Lake enforces a **minimum retention period of 7 days** for safety. Running `VACUUM` with `RETAIN 0 HOURS` will fail unless you explicitly disable the retention check using:

```
1 | CopyEditSET spark.databricks.delta.retentionDurationCheck.enabled = false;
```

Explanation

Changes only specific rows based on a condition, doesn't replace the entire dataset.

Correct answer

☐ **INSERT OVERWRITE**

Explanation

1. **INSERT OVERWRITE** replaces **all existing records** in a Delta table with new data in a **single, efficient operation**.
2. It is ideal for **full refresh** scenarios, like batch jobs that generate a complete dataset every time.

☐ **ALTER TABLE**

Databricks Repos are primarily used for version control of notebooks and other files, not for managing privileges.

Databricks Repos supports creating and managing branches for development work.

Commit & Push is used to save the changes on a local repo, then uploads this local repo content to the remote repository.

Databricks Repos supports git Pull operation

Your answer is correct

☒ **Databricks repos allow you to comment and commit code changes and push them to a remote branch**

Explanation

This is correct. Databricks Repos supports Git operations including commit, comment, and push to remote branches.

Overall explanation

A securable object is an object defined in the Unity Catalog metastore on which privileges can be granted to a principal (user, service principal, or group). Securable objects in Unity Catalog are hierarchical.

The securable objects are:

- **METASTORE**
- **CATALOG**
- **SCHEMA**
- **TABLE**
- **VIEW**
- **MATERIALIZED VIEW**
- **VOLUME**
- **FUNCTION**
- **Model**
- **EXTERNAL LOCATION**
- **SERVICE CREDENTIAL**
- **STORAGE CREDENTIAL**
- **CONNECTION**
- **SHARE**
- **RECIPIENT**
- **PROVIDER**
- **CLEAN ROOM**

Resources

[🔗 Unity Catalog privileges and securable objects](#)

Commands:

Read:

```
SELECT * FROM file_format.`/path/file/`
```

We can query older versions of the table. This can be done in 2 different ways:

- Using a timestamp
 - `SELECT * FROM my_table TIMESTAMP AS OF "2024-07-07"`
- Using a version number
 - `SELECT * FROM my_table VERSION AS OF 36`
 - `SELECT * FROM my_table@v36`

MEDALIION:

process of bringing the data into the system. Once ingested, the data would be stored in the **Bronze table** (not "used" by it). This job involves bringing data into the Lakehouse, but it doesn't directly **use** the Bronze table in the typical sense of processing or transforming data.

- ☐ A job that queries aggregated data to publish key insights into a dashboard

Explanation

The job here requires **aggregated data**, which means the data has already gone through transformations or summarizations, likely from **Silver** or **Gold tables**. The **Bronze table** contains raw, untransformed data, so it's not suitable for direct querying for a dashboard, as the data would need to be cleaned and processed first.

- ☐ A job that develops a feature set for a machine learning application

Explanation

Developing features for a machine learning application typically involves working with **cleaned and transformed data** to generate useful features for models. This process usually starts with **Silver** or **Gold** data, which has already been preprocessed. Raw data from the **Bronze table** often requires substantial cleaning and transformation before it's suitable for machine learning feature engineering.

Correct answer

- ☐ A job that enriches raw data by parsing its timestamps into a human-readable format

Explanation

This job **directly operates on raw data** (such as parsing timestamps), and the **Bronze table** contains that raw, untransformed data. By enriching raw data with transformations like timestamp parsing, the job works directly with the Bronze table before it's processed or moved to more refined layers. This makes **E** the most fitting choice, as it's transforming raw data, which is exactly what the Bronze table holds.

- ☐ A job that aggregates cleaned data to create standard summary statistics

Explanation

Aggregation typically happens on **cleaned or processed data**, which is found in the **Silver** or **Gold tables**. The Bronze table contains raw, unprocessed data, which is usually not yet suitable for aggregation or summary statistics without additional cleaning or transformation. This job would more likely start with data from a **Silver** table, where the data is cleaner and ready for aggregation.

The lakehouse architecture replaces the dependency on both data lakes and warehouses by combining their benefits and supporting open standards, BI workloads, and machine learning needs.

Incorrect. COMMENT cannot precede the CREATE TABLE statement standalone.

Your answer is incorrect

- ☒ `CREATE TABLE payments AS SELECT * FROM bank_transactions COMMENT("This table contains sensitive information")`

Explanation

Incorrect. Incorrect position and syntax of COMMENT.

Correct answer

- ☐ `CREATE TABLE payments COMMENT "This table contains sensitive information" AS SELECT * FROM bank_transactions`

Explanation

Correct. This is the correct syntax: COMMENT followed by quoted text before AS SELECT.

- ☐ `CREATE TABLE payments AS SELECT * FROM bank_transactions COMMENT "This table contains sensitive information"`

Explanation

Incorrect. COMMENT must be placed before AS SELECT, not after.

☐ Storage

Explanation

Incorrect. Unity Catalog does not manage permissions directly at the storage level; it governs access at a higher level.

Correct answer

☐ Account

Explanation

Correct. Unity Catalog manages permissions at the account level and provides centralized governance across all workspaces.

Overall explanation

Unity Catalog operates at the account level, enabling centralized governance and access control across multiple Databricks workspaces.

Explanation

Incorrect. The correct reference to another live table should include the LIVE. prefix.

Correct answer

- ☐ **CREATE LIVE TABLE aggregated_sales AS SELECT store_id, sum(total) FROM LIVE.cleaned_sales GROUP BY store_id**

Explanation

Correct. This syntax properly uses both CREATE LIVE TABLE and references another live table with the LIVE. prefix.

Your answer is incorrect

- ☒ **CREATE STREAMING TABLE aggregated_sales AS SELECT store_id, sum(total) FROM LIVE.cleaned_sales GROUP BY store_id**

Explanation

Incorrect. STREAMING TABLE is not a valid keyword combination for DLT SQL syntax.

- ☐ **CREATE STREAMING LIVE TABLE aggregated_sales AS SELECT store_id, sum(total) FROM STREAM(cleaned_sales) GROUP BY store_id**

Explanation

Incorrect. The STREAM function is not necessary when referencing another live table in DLT SQL.

Overall explanation

In DLT pipelines, 'CREATE LIVE TABLE' must be used to define new tables, and referenced tables must be prefixed with 'LIVE.' to be recognized correctly.

Domain

Data Processing

STREAMING VS DLT vs LIVE tables

Question 29 **Incorrect**

How do you check the location of an existing schema in Delta Lake?

Correct answer

- ☐ Run SQL command DESCRIBE SCHEMA EXTENDED schema_name

Explanation

Correct. DESCRIBE SCHEMA EXTENDED returns the location of the schema along with other metadata.

Your answer is incorrect

- ☒ Run SQL command SHOW LOCATION schema_name

Explanation

Incorrect. There is no SQL command called SHOW LOCATION for schemas.

Explanation

Incorrect. VACUUM removes files but does not necessarily compact data. Compaction requires the OPTIMIZE command.

Your answer is correct

- ✔ **Command will fail, you cannot run the command with retentionDurationCheck enabled**

Explanation

Correct. Delta Lake prevents the execution of VACUUM with a 0-hour retention due to retentionDurationCheck, which avoids unintentional data loss.

✔ Question 3 Correct

Kevin is the owner of both the sales table and regional_sales_vw view which uses the sales table as the underlying source for the data, and Kevin is looking to grant select privilege on the view regional_sales_vw to one of newly joined team members Steven. Which of the following is a true statement?

☐ Kevin can not grant access to Steven since he does not have security admin privilege

Explanation

Incorrect. Ownership of the view and table allows Kevin to grant access without needing security admin privileges.

☐ Steve will also require SELECT access on the underlying table

Explanation

Incorrect. Since the permission is granted on the view and Kevin owns both the view and table, no additional permission is required for Steven on the table.

☐ Kevin although is the owner but does not have ALL PRIVILEGES permission

Explanation

Incorrect. Being the owner of an object in Databricks implicitly includes the privilege to grant permissions like SELECT.

Your answer is correct

✔ Kevin can grant access to the view, because he is the owner of the view and the underlying table

Explanation

Correct. Ownership of both the view and the table allows Kevin to grant SELECT privilege on the view.

testing, which of the following best describes the execution modes of this DLT pipeline?

✓ Question 17 Correct

One of the team members, Steve, who has the ability to create views, created a new view called `regional_sales_vw` on the existing table called `sales` which is owned by John. Another team member, Kevin, wanted to query the data in `regional_sales_vw`, so Steve granted permission using the command: `GRANT VIEW, USAGE ON regional_sales_vw TO kevin@company.com`. But Kevin is still unable to access the view. Why?

☐ Kevin needs select access on the table `sales`

Explanation

Incorrect. Kevin needs access, but Steve must have ownership to grant it.

☐ Kevin is not the owner of the sales table

Explanation

Incorrect. Ownership is not required to read a table or view, only appropriate permissions.

Your answer is correct

☒ Steve is not the owner of the sales table

Explanation

Correct. Since Steve is not the owner of the base table, he cannot grant access to derived objects based on it.

☐ Table access control is not enabled on the table and view

Explanation

Incorrect. Table access control must be enabled, but that is not the root cause described.

☐ Kevin needs owner access on the view `regional_sales_vw`

Explanation

Incorrect. Kevin does not need to be the owner, just needs the correct access granted by an owner.

Overall explanation

In Databricks, only the owner of a table can grant privileges on it or on views derived from it. Since Steve is not the owner of the `sales` table, he cannot grant Kevin access to a view based on that table.

In Databricks, only the owner of a table can grant privileges on it or on views derived from it. Since Steve is not the owner of the sales table, he cannot grant Kevin access to a view based on that table.

The silver layer in the Medallion architecture refines raw data from the bronze layer by removing duplicates, enforcing schema, and performing data quality checks.

Correct. Auto Loader supports directory listing to scan input directories and file notifications to track new files efficiently.

Checkpointing and watermarking help in structured streaming

Checkpointing and write-ahead logging ensure fault tolerance and offset

STREAM VIEW:

creation of a view on top of a Delta stream (stream on a Delta table)?

```
Spark.readStream.format("delta").table("sales").createOrReplaceTempView("streaming_vw")
```

Retention:

```
ALTER TABLE table_name set TBLPROPERTIES (delta.deletedFileRetentionDuration= 'interval 365 days')
```

Autoloader:

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "parquet")
  .option("cloudFiles.schemaLocation",
    "dbfs:/mnt/demo/orders_checkpoint")
  .load(f"{dataset_bookstore}/orders-raw")
  .writeStream
    .option("checkpointLocation", "dbfs:/mnt/demo/orders_checkpoint")
    .table("orders_updates"))
```

)

```
SET spark.sql.files.ignoreMissingFiles = true; RESTORE TABLE employees1 TO  
VERSION AS OF 4
```